

Who's the (Multi-)Fairest of Them ALL: Rethinking Interpolation-Based Data Augmentation Through the Lens of Multicalibration

Karina Halevy,^{1,2} Karly Hou,² Charumathi Badrinath²

¹Carnegie Mellon University

²Harvard University

khalevy@andrew.cmu.edu

Abstract

Data augmentation methods, especially SoTA interpolation-based methods such as Fair Mixup, have been widely shown to increase model fairness. However, this fairness is evaluated on metrics that do not capture model uncertainty and on datasets with only one, relatively large, minority group. As a remedy, multicalibration has been introduced to measure fairness while accommodating uncertainty and accounting for multiple minority groups. However, existing methods of improving multicalibration involve reducing initial training data to create a holdout set for post-processing, which is not ideal when minority training data is already sparse. This paper uses multicalibration to more rigorously examine data augmentation for classification fairness. We stress-test four versions of Fair Mixup on two structured data classification problems with up to 81 marginalized groups, evaluating multicalibration violations and balanced accuracy. We find that on nearly every experiment, Fair Mixup *worsens* baseline performance and fairness, but the simple vanilla Mixup *outperforms* both Fair Mixup and the baseline, especially when calibrating on small groups. *Combining* vanilla Mixup with multicalibration post-processing, which enforces multicalibration through post-processing on a holdout set, further increases fairness.

Code — <https://github.com/ENSCMA2/fairest-mixup>

Proceedings version —

<https://ojs.aaai.org/index.php/AAAI/article/view/33870>

1 Introduction

Algorithmic fairness has become increasingly important with the ubiquitous application of machine learning (ML). Unfairness can arise from many sources (Huang et al. 2022), including unequal representation of protected groups in data (Guo et al. 2022). For example, people of color can be underrepresented in clinical trials due to access barriers, lack of information, and discrimination (Allison, Patel, and Kaur 2022), leading ML models to have trouble predicting treatment outcomes for non-white patients. One way to mitigate underrepresentation is data augmentation, which creates synthetic individuals from the original data (Chuang and Mroueh 2021; Iosifidis and Ntoutsis 2018; Chawla et al. 2002; Sharma et al. 2020). A particularly promising

form of augmentation is Mixup (Zhang et al. 2017) and its fairness-oriented counterpart Fair Mixup (Chuang and Mroueh 2021), which linearly interpolate individuals with features in between majority and minority group attributes. However, existing augmentation literature measures fairness through binary metrics like demographic parity and equalized odds (Chuang and Mroueh 2021), which accumulate loss even when predictors lean toward correct labels. These metrics can be misleading because data often does not include all predictive features, so some notion of uncertainty is appropriate in a good predictor but would be penalized. Furthermore, the methods in Chuang and Mroueh (2021) only assess and optimize fairness for one minority group, but a fair predictor should work well on multiple multi-dimensional intersecting groups.

The metric of multicalibration (MC) (Hebert-Johnson et al. 2018) accounts for this uncertainty and for the presence of multiple groups by comparing predicted probabilities to true probabilities, averaging over groups of interest, and considering subsets of a predictor's support separately. Hebert-Johnson et al. (2018) also introduce an algorithm, with runtime inversely proportional to the size of the smallest group, to post-process a predictor using a holdout set and guarantee a maximum MC violation. Barda et al. (2020) then use this algorithm to learn prediction adjustments from a holdout set and apply those adjustments to test predictions. However, such post-processing subtracts a substantial amount of holdout data from available training data, resulting in even less representation of underrepresented groups in initial training. Moreover, with runtime inversely proportional to group size, enforcing MC for very small groups can be slow. The guarantees of MC enforcement and upper bounds of the overall accuracy tradeoffs proven in Hebert-Johnson et al. (2018) also only apply to the post-processed holdout set, not to unseen test data.

This work examines whether we can combine the desirable properties of MC and data augmentation to supplement the binary outcome insights that demographic parity and equalized odds provide. We ask:

1. RQ1: Under what conditions can Fair Mixup mitigate MC violations of neural network predictors on minority groups while preserving binary classification accuracy?
2. RQ2: When can (Fair) Mixup serve as an alternative to and/or increase the efficiency of MC post-processing?

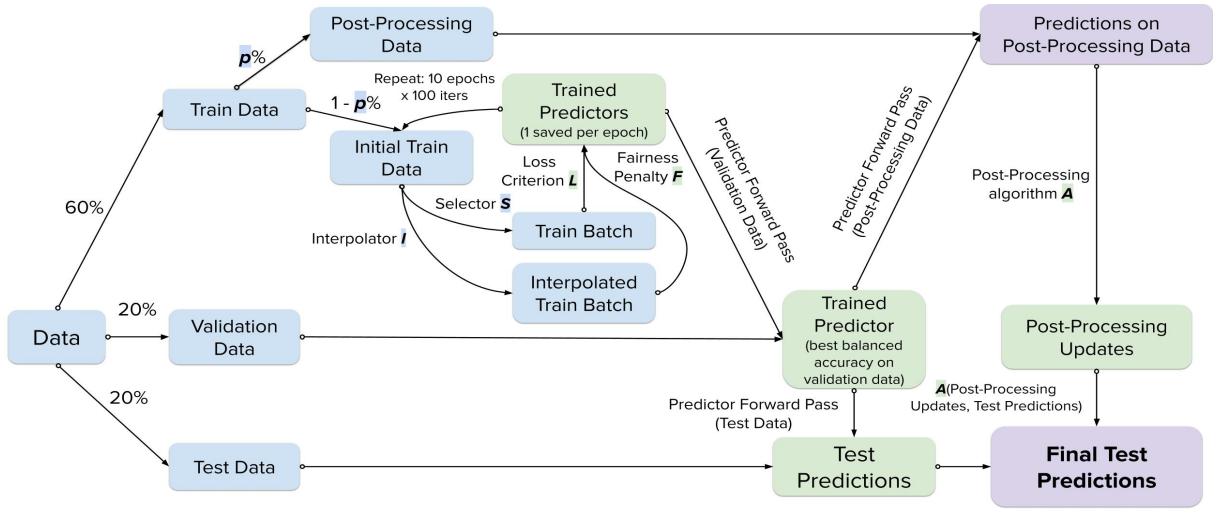


Figure 1: The ML training and evaluation pipelines considered in our work. Each method in our experiments can be characterized by a unique combination of: a percentage p of post-processing data taken from training data, an interpolation-based data augmentation method I , a training batch selection procedure S , a training loss criterion L , a training fairness penalty F , and a post-processing algorithm A . These unique combinations are listed in Table 1.

3. RQ3: What aspects of Fair Mixup contribute to its success or failure in improving MC-based fairness?

We contribute the first MC-based investigation of several (Fair) Mixup- and MC-inspired neural network training methods (depicted in Figure 1), stress-testing performance and fairness on intersecting demographic groups and creating a new perspective on whether data augmentation is effective. We find that Fair Mixup can only mitigate MC violations and outperform post-processing under its original design of optimizing one group at a time. However, vanilla Mixup consistently makes predictors fairer and results in an average balanced accuracy/MC violation improvement of up to 14.22% when combined with MC post-processing. We also find that the key performance-enhancing component of Fair Mixup is that it learns from interpolated data points. However, its other components (balancing training data by minority group membership and penalizing pairwise unfairness during training) detract from baseline performance, resulting in average balanced accuracy/MC violation decreases of up to 12.29%.

2 Preliminaries

This section defines calibration (Hebert-Johnson et al. 2018; Chouldechova 2017), multicalibration (Hebert-Johnson et al. 2018), multiaccuracy (Hebert-Johnson et al. 2018), and the data augmentation methods we later expand on.

2.1 Notation

Throughout this paper, $\mathcal{X}$ represents a universe of individuals, x_i represents an individual with index i , $S \subseteq \mathcal{X}$ is a subset of individuals, $\mathcal{C} \subseteq 2^{\mathcal{X}}$ is a set of subsets of individuals, f is a predictor that maps individual x_i to outcome probability f_i , p_i^* is the true outcome probability of x_i , and $y_i \in \{0, 1\}$ is the binarized true outcome for x_i .

2.2 Calibration

For a maximum violation $\alpha \in [0, 1]$, f is α -**calibrated** w.r.t. S if $\exists S' \subseteq S$ with $|S'| \geq (1 - \alpha)|S|$ such that $\forall v \in [0, 1]$,

$$|\mathbb{E}_{x_i \sim (S_v \cap S')} [f_i - p_i^*]| \leq \alpha, \quad (1)$$

where $S_v = \{x_i : f_i = v\}$. In most classification tasks, we only see the binary outcome y_i for x_i . Thus, we use a modification called **observable** calibration (Hebert-Johnson et al. 2018), where y_i replaces p_i^* in Eq. 1.

For example, a tumor malignancy classifier is 0.05-observably calibrated for $v = 0.6$ on Latine patients if of all Latine patients for which it predicts a 60% chance of malignancy, 55% to 65% of these patients have a malignant tumor. The classifier is 0.05-observably calibrated on Latine patients if this holds for all v —of all Latine patients for which it predicts a v chance of malignancy, between $v - 5\%$ and $v + 5\%$ of these patients have a truly malignant tumor.

2.3 Multicalibration

f is $(\mathcal{C}, \alpha)$ -**multicalibrated** if it is α -calibrated w.r.t. all $S \in \mathcal{C}$ (Hebert-Johnson et al. 2018). We define MC as in Hebert-Johnson et al. (2018), but we require $S = S'$ (calibration on all of S rather than any $1 - \alpha$ of it, explained in Appendix C). For computational feasibility over datasets with millions of prediction probabilities, we also discretize the predicted probabilities. For integer $d > 0$, the d -**discretized** version of S splits S into $d + 1$ subsets, where

$$S_v = \{x_i : \frac{v}{d} \leq f_i < \frac{v+1}{d}\} \text{ for } v \in [0, 1, \dots, d]. \quad (2)$$

Continuing with the tumor malignancy classifier example, the subset of the 10-discretized $S = \text{Latine patients}$ with $v = 6$ would be all Latine patients with a predicted chance of at least 60% but less than 70% malignancy. Suppose all patients in this subset have a prediction of 63%.

0.05-calibration would require that 58 to 68% of these patients have a truly malignant tumor, and that the corresponding conditions hold for all other $v \in [0, 10]$. Given $\mathcal{C} = \{\text{Black patients, Asian patients, Latine patients}\}$, $\mathcal{C}, 0.05$ -multicalibration requires that this 0.05-calibration must hold for Black, Asian, and Latine patients.

2.4 Multiaccuracy

Multiaccuracy (MA) (Hebert-Johnson et al. 2018) is a looser version of MC. f is $(\mathcal{C}, \alpha)$ -**multiaccurate** if $\forall S \in \mathcal{C}$,

$$|\mathbb{E}_{x_i \sim S}[f_i - p_i^*]| \leq \alpha. \quad (3)$$

Rather than requiring the expected prediction error within each S and predicted probability to be $\leq \alpha$, MA only requires this error to be $\leq \alpha$ in S overall. Thus, for $\mathcal{C} = \{\text{Black patients, Latine patients}\}$, $(\mathcal{C}, 0.05)$ -multiaccuracy means that the average prediction for Black patients is within 5% of the true proportion of Black patients that have a malignant tumor, and likewise for Latine patients. In the rest of this paper, when we say f has an MC or MA violation of α on $\mathcal{C}$, we mean that α is the smallest value for which f is $(\mathcal{C}, \alpha)$ -multicalibrated or multiaccurate.

2.5 Mixup

Mixup was proposed to improve the generalizability of neural networks (NN) by training on linear combinations of example pairs, with the intuition that the NN would learn how predictions differ as inputs move continuously between feature sets (Zhang et al. 2017). For training batch size b , mixup draws $(x_1, y_1), \dots, (x_b, y_b)$ and $(x'_1, y'_1), \dots, (x'_b, y'_b)$ without replacement from the training data. Let $t \sim \text{Beta}(\epsilon, \epsilon)$ where $\epsilon \in (0, \infty)$. Mixup constructs one synthetic point per $i \in [1, \dots, b]$:

$$(x''_i, y''_i) = (tx_i + (1-t)x'_i, ty_i + (1-t)y'_i) \quad (4)$$

and trains an NN on $(x''_1, y''_1), \dots, (x''_b, y''_b)$ instead of the original batch. Zhang et al. (2017) showed that mixup decreased test error on CIFAR-10 and CIFAR-100.

2.6 Fair Mixup

Chuang and Mroueh (2021) adapted mixup toward the goal of fairness. **Fair Mixup** (FM) samples $(x_1, y_1), \dots, (x_b, y_b)$ from minority group S and $(x'_1, y'_1), \dots, (x'_b, y'_b)$ from $S' = \neg S$. Mixup is then performed on these samples as in Section 2.5 to create synthetic points. The loss function applies the standard Binary Cross Entropy (BCE) loss function to the original points, applies the gradient $\mathcal{R}_{\text{mixup}}^{\mathcal{M}_S}$ of a pairwise fairness penalty $\mathcal{M}$ between S and S' to the synthetic points, and adds λ times the fairness penalty to the BCE. Fair Mixup creates better tradeoffs between average precision and the fairness metrics of demographic parity and equalized odds (Chuang and Mroueh 2021).

3 Related Work

3.1 Data Augmentation for Fairness

There are several other data augmentation methods for fairness. In oversampling, minority group samples are duplicated until equal in number to majority group samples (Iosifidis and Ntoutsi 2018). Another method, SMOTE, creates

minority group members through linear interpolation among existing minority group members (Chawla et al. 2002). More recently, Sharma et al. (2020) introduce “Ideal World”: for each original point, a new sample is created with the same features and label, but the protected attribute is flipped, making both statistical parity difference and average odds difference decrease while preserving accuracy. Outside of structured data, Wadhwa et al. (2022) apply identity pair replacement, identity term blindness, and identity pair swap on text classification. Yucer et al. (2020) introduce data augmentation that improves facial recognition on minority groups.

We focus on structured data classification to minimize the confounding factor of unstructured data featurization. We also choose Fair Mixup as a basis because it minimizes data distribution changes and treats protected attributes as predictive features. Ideal World takes away the predictive information of protected attributes. Oversampling, SMOTE, and Ideal World create additional minority individuals, changing the frequency and composition of minority groups. In contrast, Fair Mixup creates individuals that are neither minority nor majority group members, but rather some interpolated in-between. Thus, while the data distribution may change, the members of boolean circuit-defined groups do not.

3.2 Extensions of MC

Hebert-Johnson et al. (2018) devise algorithms that could enforce MC α ’s to be below an arbitrary threshold. A related post-processing algorithm, designed for multiaccuracy, is MULTIACCURACY BOOST, which requires a trained auditor on top of a holdout set (Kim, Ghorbani, and Zou 2019). Applying the results of Hebert-Johnson et al. (2018) empirically, Barda et al. (2020) transfer learned post-processing updates to a COVID-19 mortality rate forecasting task. We test this application in Section 4.4.

A few works extend (multi-)calibration to more nuanced metrics that handle complex notions of uncertainty. Kumar, Sarawagi, and Jain (2018) add calibration optimization to the training loss function, clamping overconfident predictions while minimizing penalties on true confident predictions. Wald et al. (2021) propose multi-domain calibration to evaluate model generalization to out-of-distribution data, suggesting both isotonic regression post-processing and a training regime that includes calibration from Kumar, Sarawagi, and Jain (2018). Jung et al. (2021) extend MC to higher moments, measuring moment consistency in a way that computes groupwise error inversely proportionally to group size (Jung et al. 2021). Other work extends MC to conformal prediction, which generates prediction sets rather than point estimates (Jung et al. 2023; Foygel Barber et al. 2020). This framework generalizes MC to quantiles of the label’s support rather than individual values and is useful for categorical or continuous labels, unlike binary labels, for which MC is already a probabilistic extension. Gopalan et al. (2024) connect MC to multi-group loss minimization.

The most comprehensive investigation of MC post-processing to our knowledge is Hansen et al. (2024), which finds that baseline predictors on tabular data are often decently multicalibrated already, and post-processing does not improve worst-group calibration error for multi-layer per-

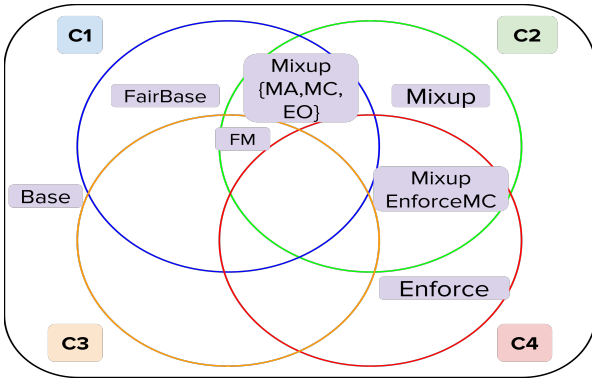


Figure 2: Venn Diagram of each method’s core components.

ceptrons, Random Forests, and Logistic Regression but does benefit Support Vector Machines, Decision Trees, and Naive Bayes. When worst-group calibration error improves, there is an overall accuracy tradeoff. They further find that MC enforcement is hyperparameter-sensitive and most effective with huge amounts of data (found in image and language data but not tabular data). They find that calibration algorithms like Platt scaling and isotonic regression sometimes perform nearly on par with MC enforcement while being more efficient. These findings are consistent with previous works suggesting that empirical risk minimization may inevitably yield multicalibrated baseline predictors (Błasiok et al. 2023, 2024). We refer the reader to Hansen et al. (2024) for a more comprehensive MC literature review and for image and language experiments.

This work extends Hansen et al. (2024) in three ways. First, expanding upon their maximum of 15 groups that are all at least 0.5% of their corresponding population, we stress-test our methods on MC w.r.t. up to 81 groups at a time, up to 55 of which are smaller than 0.25% of their corresponding population. We also select these groups in five different ways to investigate effects of group set size on MC. Second, expanding upon their examination of income prediction from `folktables` on Californian residents from 2018, we evaluate our methods on each permutation of the 10 most populous US states and the four most recent American Community Survey data collection years, yielding 40 datasets. We additionally test employment status prediction on these 40 datasets, for a total of 80 tasks considered. Third, while their work and much of the current MC literature considers data-reductive post-processing methods, our work takes inspiration from their finding that post-processing works best on huge datasets and instead focuses on data augmentation to maximize the amount of original data that can be used for initial training.

4 Methods

We test 13 NN training methods to determine the effects of particular features of FM that contribute to its performance and fairness. FM has 3 distinguishing components:

1. C1: Training batches are balanced across membership in the minority group for which we wish to ensure fairness.

2. C2: Synthetic data is created by linearly interpolating original points. If C1 is implemented, each synthetic data point is the interpolation of a minority group member and a majority group member. If not, the original points are split in half and paired at random for interpolation.
3. C3 (can only be done if C2 is also implemented): A fairness penalty is added to the loss function for predictions on synthetic points, minimizing a weighted sum of the standard loss and the fairness penalty during training.

Post-processing is distinguished by the following:

4. C4: A post-processing algorithm learns prediction update rules from post-processing data (subtracted from initial training data), and it applies those update rules to the validation and test data during evaluation and deployment.

With these insights (summarized in Fig. 2), this section describes each method mathematically. We motivate each method by explaining how it implements a subset of $\{C1, C2, C3, C4\}$, thus isolating the effects of specific components of FM to answer RQ3. C4 also helps answer RQ2 (FM vs. post-processing). Method names are starred if they contain substantial novel elements that we introduce on top of existing work. Implementation details are in Appendix C.

4.1 Baselines

BASE trains an NN with mini-batch gradient descent using Binary Cross Entropy loss, over several epochs and batch selection iteration. We report test-time balanced accuracy for the epoch with the best validation-time balanced accuracy. BASE does not implement C1, C2, C3, or C4.

***FAIRBASE** modifies BASE by balancing training data groupwise (C1). Suppose we have minority groups $\mathcal{C}$ to optimize for fairness and n iterations of gradient descent per training epoch in BASE. Then, FAIRBASE conducts $n \cdot |\mathcal{C}|$ iterations of gradient descent. Each iteration centers around one $S \in \mathcal{C}$: we construct a batch by selecting one sub-batch from S and one sub-batch from its complement $\neg S$. We sub-sample the larger sub-batch to be equal in size to the smaller sub-batch to ensure balance across membership in S .

4.2 Variants of Mixup

MIXUP is as defined in Section 2.5, implementing C2.

***MIXUP_{EO}** modifies FAIRBASE. Consider minority groups $\mathcal{C}$ and $n \cdot |\mathcal{C}|$ iterations of gradient descent as in FAIRBASE. MIXUP_{EO} conducts $2n \cdot |\mathcal{C}|$ iterations of gradient descent, each centered around one pair $(S, y) \in (\mathcal{C}, \{0, 1\})$. We construct a batch by selecting one sub-batch of members of S_y (members of S whose true label is y) and one sub-batch of members of S'_y (members of $\neg S$ whose true label is y). Next, we perform mixup by pairing each member of S_y with a member of S'_y within the batch and interpolating each pair. Our loss is a weighted sum of Binary Cross Entropy applied to the original batch and the same loss applied to the interpolated points. MIXUP_{EO} implements C1, C2, and a control version of C3 (standard loss instead of pairwise fairness, but number of groups under consideration for this loss is adjustable, as elaborated on in Section 4.3). Thus, we can compare it to FAIRBASE to isolate the effect of C2.

Method	p	I	S	L	F	A
BASE	0	$I(\cdot) = \emptyset$	uniform random	BCE	$F(\cdot) = 0$	$A(\cdot) = \emptyset$
FAIRBASE	0	$I(\cdot) = \emptyset$	balance by group	BCE	$F(\cdot) = 0$	$A(\cdot) = \emptyset$
MIXUP	0	Mixup	uniform random	$L(\cdot) = 0$	BCE	$A(\cdot) = \emptyset$
MIXUP _{EO}	0	Mixup	balance by group $\times y_i$	BCE	$\lambda \cdot \text{BCE}$	$A(\cdot) = \emptyset$
MIXUP _{MA}	0	Mixup	balance by group	BCE	$\lambda \cdot \text{BCE}$	$A(\cdot) = \emptyset$
MIXUP _{MC}	0	Mixup	balance by group $\times f_i$	BCE	$\lambda \cdot \text{BCE}$	$A(\cdot) = \emptyset$
FM _{DP}	0	Mixup	balance by group	BCE	$\lambda \cdot \mathcal{R}_{\text{mixup}}^{\text{DP}}$	$A(\cdot) = \emptyset$
FM _{EO}	0	Mixup	balance by group $\times y_i$	BCE	$\lambda \cdot \mathcal{R}_{\text{mixup}}^{\text{EO}}$	$A(\cdot) = \emptyset$
FM _{MA}	0	Mixup	balance by group	BCE	$\lambda \cdot \mathcal{R}_{\text{mixup}}^{\text{MA}}$	$A(\cdot) = \emptyset$
FM _{MC}	0	Mixup	balance by group $\times f_i$	BCE	$\lambda \cdot \mathcal{R}_{\text{mixup}}^{\text{MC}}$	$A(\cdot) = \emptyset$
ENFORCE _{MA}	25	$I(\cdot) = \emptyset$	uniform random	BCE	$F(\cdot) = 0$	Listing 2
ENFORCE _{MC}	25	$I(\cdot) = \emptyset$	uniform random	BCE	$F(\cdot) = 0$	Listing 1
MIXUP _{ENFORCE_{MC}}	25	Mixup	uniform random	$L(\cdot) = 0$	BCE	Listing 1

Table 1: Post-processing data split percentages p , data augmentors I , training batch selectors S , loss criteria L (applied to original data), fairness penalties F (applied to synthetic data), and post-processing algorithms A that uniquely characterize each method described in Section 4 and diagrammed in Fig. 1. BCE stands for Binary Cross Entropy loss.

***MIXUP_{MA}** creates one balanced batch per $S \in \mathcal{C}$, as in FAIRBASE, yielding $n \cdot |\mathcal{C}|$ gradient descent iterations. We interpolate each batch by pairing members of S with members of $\neg S$ and adding λ times the Binary Cross Entropy on the interpolated points. MIXUP_{MA} also implements C1, C2, and a control version of C3, though C1 is slightly different than in MIXUP_{EO}, allowing us to compare variations of C1.

***MIXUP_{MC}** creates $d + 1$ batches per $S \in \mathcal{C}$ by creating d -discretized intervals of f_i 's, yielding $(d + 1) \cdot n \cdot |\mathcal{C}|$ gradient descent iterations. For each S_v^d (members of S with predicted probability in $[\frac{v}{d}, \frac{v+1}{d})$), we construct a batch with half its points from members of S_v and the other half from members of $(\neg S)_v^d$. We interpolate and calculate loss as in MIXUP_{EO}. MIXUP_{MC} also implements C1, C2, and a control version of C3, providing another way to compare the specifics of C1.

4.3 Variants of Fair Mixup

FM implements C1, C2, and C3. Though Chuang and Mroueh (2021) introduce two versions of FM (with $\mathcal{M}$ as demographic parity difference and equalized odds difference), their framework generalizes to any pairwise fairness metric. We first show how to modify FM to accommodate multiple minority groups simultaneously. Then, we define the two versions of FM from Chuang and Mroueh (2021), followed by two versions with new metrics. Specifically, our extensions try to incorporate some notion of MC in the fairness penalty, as we aim to minimize MC violations. These methods test the effects of varying the metric in C3.

Modifying (Fair) Mixup for Multiple Groups Consider metric $\mathcal{M}$, its group gradients $\mathcal{R}_{\text{mixup}}^{\mathcal{M}_{S_1}}, \dots, \mathcal{R}_{\text{mixup}}^{\mathcal{M}_{S_{|\mathcal{C}|}}}$. For $k \in \{1, \dots, |\mathcal{C}|\}$, the penalty is the mean of the k highest group gradients. We take means because preliminary experiments show that sums produce higher MC α s. We make k adjustable to prevent overfitting. The rest of the computation proceeds as in Chuang and Mroueh (2021). Given $\mathcal{M}_S$, the first step is to transform it into an integral via the Fundamental Theorem of Calculus so it can be computed for interpo-

lated data (Appendix A.1 in Chuang and Mroueh (2021)). Then, we differentiate the integral to get $\mathcal{R}_{\text{mixup}}^{\mathcal{M}_S}$. We list the equations for $\mathcal{M}_S$ below, with full formulae in Appendix D.

FM_{DP} is the first version of FM in Chuang and Mroueh (2021). $\mathcal{M}_S$ is demographic parity, the difference between the average f_i on members of S vs. its complement S' :

$$\mathcal{M}_S = \Delta \text{DP}_S(f) = |\mathbb{E}_{x_i \sim S}[f_i] - \mathbb{E}_{x_i \sim S'}[f_i]|. \quad (5)$$

FM_{EO} is the second version of FM, with the equalized odds difference (Hardt, Price, and Srebro 2016) that modifies DP by only considering one true outcome at a time:

$$\mathcal{M}_S = \Delta \text{EO}_S(f) = \sum_{y \in \{0,1\}} |\mathbb{E}_{x_i \sim S_y}[f_i] - \mathbb{E}_{x_i \sim S'_y}[f_i]|, \quad (6)$$

where $S_y = \{x_i \in S : y_i = y\}$, and $S' = \neg S$.

***FM_{MA}** is our first extension of FM, with a version of MA modified to be pairwise. We measure the mean difference in prediction errors $e_i = f_i - p_i^*$ between S and S' :

$$\mathcal{M}_S = \Delta \text{MA}_S(f) = |\mathbb{E}_{x_i \sim S}[e_i] - \mathbb{E}_{x_i \sim S'}[e_i]|, \quad (7)$$

***FM_{MC}** is our second extension of FM, with a pairwise modification of MC, which modifies MA by considering one interval $S_v^d = \{x_i \in S : f_i \in [\frac{v}{d}, \frac{v+1}{d})\}$ at a time:

$$\mathcal{M}_S = \Delta \text{MC}_S(f) = \sum_{v=0}^d |\mathbb{E}_{x_i \sim S_v^d}[e_i] - \mathbb{E}_{x_i \sim S_v'^d}[e_i]|. \quad (8)$$

4.4 Post-Processing

We test whether MC and MA enforcement (implementing C4) improve test performance as in Barda et al. (2020).

ENFORCE_{MA} post-processes predictions to minimize MA violations. We feed (1) predictions on a holdout post-processing set and (2) a set of minority groups $\mathcal{C}$ as inputs to Algorithm 3.1 in Hebert-Johnson et al. (2018). However, we augment Algorithm 3.1 with a list of rules mapping each $S \in \mathcal{C}$ to a float a_S to be added to predictions on members of

S. In other words, the algorithm learns how much to adjust predictions for each group. At validation and test time, we add a_S to initial predictor outputs for members *S*.

ENFORCE_{MC} post-processes predictions to minimize MC violations. It proceeds as in ENFORCE_{MA}, but we add an integer d as a third input to Algorithm 3.2 in Hebert-Johnson et al. (2018). We augment Algorithm 3.2 with a list of rules mapping each group S_v (members of $S \in \mathcal{C}$ where $f_i \in [\frac{v}{d}, \frac{v+1}{d})$), to a float $a_{S,v}$ to be added to predictions on members of S whose initial predictions are in $[\frac{v}{d}, \frac{v+1}{d})$.

MIXUP_{ENFORCE_{MC}} performs MIXUP on a reduced initial training set followed by ENFORCE_{MC} on a holdout post-processing set. This method tests C2 $\cup$ C4, as we ultimately find that MIXUP performs best overall among methods that do not implement C4. We implement MIXUP_{ENFORCE_{MC}} to answer the part of RQ2 that asks whether data augmentation can improve the performance of ENFORCE_{MC}.

5 Experiments

This section describes our data and experimental settings.

5.1 Datasets

We test two prediction tasks from *folktables* (Ding et al. 2021), a superset of Adult Income data (Becker and Kohavi 1996) collected from the American Community Survey. We have $p_i^* \in \{0, 1\}$, but $f_i \in [0, 1]$. Table 2 summarizes the data. Full data statistics are at <http://tiny.cc/mfm-stats>.

EMPLOYMENT The task is to predict whether an individual is employed. Table 4 specifies the exact input features.

INCOME The task is to predict whether an individual’s annual income is higher than the median income for that year in their state of residence according to Data Commons (Google 2024). Table 5 lists input features.

We run 40 datasets each for EMPLOYMENT and INCOME: the 10 most populous US states $\times$ the 4 most recent years, providing substantial geographic and temporal variation. We choose these tasks based on experiments in Jung et al. (2023). Additionally, we seek problems with reasonable baseline performance ($\geq 80\%$ balanced accuracy on CA $\times$ 2022) to focus on improving fairness on useful classifiers.

5.2 Experimental Settings

To measure the effects of $|\mathcal{C}|$ and $|S|$, we run all combinations of datasets and training methods on five settings:

ALL $\mathcal{C} = \cup \{\text{all } n \text{ computationally possible racial groups, disabled people, disabled members of each racial group}\}$. A racial group is computationally possible if for all random seeds, at least one disabled member of that group is in each of the train, validation, and test splits. $|\mathcal{C}| = 2n + 1$.

BIG $\mathcal{C} = \cup \{b \text{ racial groups each comprising } > 0.25\% \text{ of the total dataset, disabled people, disabled members of each of the } b \text{ racial groups}\}$ $|\mathcal{C}| = 2b + 1, b \ll n$.

SMALL $\mathcal{C} = \cup \{s \text{ racial groups each comprising } \leq 0.25\% \text{ of the total dataset, disabled people, disabled members of each of the } s \text{ racial groups}\}$. $|\mathcal{C}| = 2s + 1, s \ll n$.

DIS This setting is closest to what FM has already been tested on: $\mathcal{C} = \{\text{disabled individuals}\}$, so $|\mathcal{C}| = 1$.

DLFR $\mathcal{C} = \{\text{disabled people, members of the least frequent (computationally possible) racial group (LFR), and disabled members of the LFR}\}$, hence $|\mathcal{C}| = 3$.

6 Results

To capture both fairness and overall performance, we compute the mean across all 40 (state, year) pairs of the following quantities for each experiment: (1) % increase in balanced accuracy over BASE for the corresponding state, year, and task and (2) % decrease over BASE in worst (highest) individual group MC violation α . Table 3 reports these mean percentages, showing that for all (task, setting) pairs except for DIS (both tasks), (EMPLOYMENT, DLFR), and (INCOME, SMALL), MIXUP_{ENFORCE_{MC}} shows the biggest average balanced accuracy and MC α improvement. The other best methods are MIXUP_{MA} for (EMPLOYMENT, DIS), MIXUP for (INCOME, DIS) and (INCOME, SMALL), ENFORCE_{MC} for (EMPLOYMENT, DLFR) (though MIXUP_{ENFORCE_{MC}} is close), but FM_{DP} has the best α for (EMPLOYMENT, DIS) according to Table 8. If we consider only methods that perform post-processing or augmentation/data balancing (i.e. all methods except MIXUP_{ENFORCE_{MC}}), the best method is ENFORCE_{MC}, except (EMPLOYMENT, DIS) (MIXUP_{MA} was best), (INCOME, SMALL), and (INCOME, DIS) (MIXUP was best). One note is that for 28 of 40 datasets, we had $s = 0$ and thus $\mathcal{C} =$ just disabled people, so (INCOME, SMALL) results may be more characteristic of single-group calibration. We also note that all methods except for BASE had negative mean increases in balanced accuracy (up to -1.25%), so positive values in Table 3 indicate fairness improvements.

Examining FM, we see that except (EMPLOYMENT, DIS), all FM variants worsened fairness. For (EMPLOYMENT, DIS), FM improved fairness while largely preserving balanced accuracy, confirming the result in Chuang and Mroueh (2021) that FM works on one larger group.

Comparing MIXUP_{ENFORCE_{MC}} and ENFORCE_{MC}, we observe that while MIXUP_{ENFORCE_{MC}} outperforms ENFORCE_{MC} in many cases, it sometimes makes the ENFORCE_{MC} component of MIXUP_{ENFORCE_{MC}} less efficient. On the EMPLOYMENT dataset, the number of iterations to convergence of the ENFORCE_{MC} post-processing algorithm increased by a percentage in the range (0.34%, 5.8%), with the greatest percentage increase for SMALL (+5.8%) and the greatest decrease for BIG (-1.58%). For INCOME, all methods took fewer iterations, in the range (-3.68%, -0.08%).

Finally, we analyze correlations between results and data statistics. We largely find either no correlation or low correlations, with some exceptions. One exception is that the mean MC α across groups $> 0.25\%$ of the population on ALL has a moderate correlation with total dataset size (lower violations for bigger datasets) for all non-BASE methods

Dataset	Size	# Features (Binary, Categorical, Continuous)	# Non-White	# Disabled	Max Minority Groups	#	Max # Groups $\leq$ 0.25% of Population	Mean of Smallest Group	Size
EMPLOYMENT	6,993,839	5, 9, 2	2,160,161	1,036,251	81	55		28.5	
INCOME	3,543,292	2, 3, 6	1,014,632	250,074	51	28		31.85	

Table 2: Summary statistics of the EMPLOYMENT and INCOME datasets. “Size” is the number of individuals summed over all 40 subsets. Maxes and means are taken over these subsets. “Smallest Group” disabled members of the LFR.

Method	EMPLOYMENT					INCOME				
	ALL	BIG	SMALL	DIS	DLFR	ALL	BIG	SMALL	DIS	DLFR
BASE	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
FAIRBASE	-2.69	-2.36	-4.28	2.08	-3.76	-3.04	-4.26	-11.08	-12.29	-6.34
MIXUP	2.89	3.11	2.56	-23.20	2.03	1.22	1.30	3.12	3.20	1.63
MIXUP _{EO}	-2.54	-2.21	-4.66	4.45	-4.07	-3.39	-4.14	-9.73	-12.09	-6.15
MIXUP _{MA}	-3.31	-3.08	-4.70	4.50	-5.35	-3.10	-3.51	-11.51	-10.80	-7.27
MIXUP _{MC}	-2.91	-2.55	-5.30	3.09	-5.93	-3.08	-3.02	-10.34	-10.78	-6.58
FM _{DP}	-3.41	-2.22	-5.23	3.84	-5.29	-3.34	-3.39	-10.75	-10.11	-7.67
FM _{EO}	-2.26	-3.36	-4.05	1.73	-4.52	-3.55	-3.58	-10.00	-11.12	-5.55
FM _{MA}	-2.95	-3.99	-6.35	1.40	-5.79	-3.74	-3.41	-10.33	-11.92	-7.36
FM _{MC}	-2.72	-2.36	-4.94	0.61	-4.12	-4.32	-3.23	-10.41	-11.30	-5.52
ENFORCE _{MA}	-0.46	0.02	0.46	0.43	-0.32	-0.02	0.70	-1.27	-2.76	-1.66
ENFORCE _{MC}	8.31	12.03	7.89	-15.28	8.74	9.87	9.97	-6.61	-17.82	8.28
MIXUP _{ENFORCE_{MC}}	10.36	12.97	9.23	-29.27	8.72	11.64	14.22	-2.15	-11.37	13.06

Table 3: Summary (mean of 10 trials) of methods across 40 (state, year) pairs $\times$ EMPLOYMENT and INCOME. Each number is the mean of the % increase in balanced accuracy and % decrease in worst-group MC α .

that do not involve ENFORCE_{MC}. This mean α on ALL also moderately correlates with the number of groups bigger than 0.25% of the population for FM_{MA}, and it also moderately correlates with number of groups smaller than 0.25%, total number of minority groups, number of disabled individuals, and number of non-white individuals for several methods. Looking at efficiency, the number of iterations to convergence of ENFORCE_{MC} and MIXUP_{ENFORCE_{MC}} both strongly correlate with dataset size, but there is no correlation with the % change in number of iterations between methods.

7 Discussion

Our results reveal the importance of stress-testing fairness optimization on multiple groups of varying sizes and on metrics that capture uncertainty. To answer RQ1, **the only condition under which FM improves MC is the condition it was designed for:** fairness for one minority group (DIS) on a truly binary problem (EMPLOYMENT). This holds irrespective of the particular train-time fairness penalty. This leads to an answer to RQ2: under a **single-group truly-binary condition**, FM (especially FM_{DP}) outperforms post-processing in ensuring fairness for disabled people. Based on the raw α s in Table 8, this could be because disabled people are a relatively big, non-monolithic group for which the BASE NN is already much more calibrated than the more fine-grained racial groups. Thus, to further improve upon the BASE α , it may be more effective to examine more disabled individuals and their full feature sets during training (as in FM) rather than apply a fixed adjustment to disabled individuals unconditionally (as in post-processing). However, this fixed post-processing adjustment may work well for smaller

racial groups because the smaller sizes of racial groups make race more informative than disability.

Regular MIXUP presents a robust alternative to ENFORCE_{MA} in nearly all settings and to ENFORCE_{MC} when considering one group in a continuous-to-binary prediction problem as in (INCOME, DIS) or part of (INCOME, SMALL). More powerfully, **combining MIXUP and ENFORCE_{MC} through MIXUP_{ENFORCE_{MC}} enhances performance** of ENFORCE_{MC} alone in the majority of settings, especially when more than one group is under consideration. However, it is inconclusive whether this enhancement is accompanied by efficiency improvement for the ENFORCE_{MC} component.

For RQ3, we see that **MIXUP is the overall best post-processing-free method**. Comparing MIXUP with MIXUP_{EO}, MIXUP_{MA}, MIXUP_{MC}, and FAIRBASE, we observe that **using interpolated data contributes more to fairness improvements** than groupwise balancing of training batches. Looking at FAIRBASE, MIXUP_{EO}, MIXUP_{MA}, and MIXUP_{MC}, we further suggest that **data balancing may adversely affect performance and fairness**, since the key factor that sets MIXUP apart from worse-performing methods of FM, MIXUP_{EO}, MIXUP_{MA}, and MIXUP_{MC} is C1. This may be because having limited minority instances means we learn less about majority instances as well (and since groups intersect, some instances that are minorities in one way but majorities in another are seen less). Finally, comparing MIXUP_{EO}, MIXUP_{MA}, and MIXUP_{MC} to FM variants, we see that C3 effects (train-time fairness penalty) are inconclusive, as outcomes fluctuate by method and setting. Thinking more generally about why MIXUP outperforms FM so often, we hypothesize that in addition to the adverse effect of data balancing in FM, MIXUP has a more manageable amount of

learning (normal BCE loss, with more data to learn from, net positive), while the pairwise fairness component of FM loss may be differently valued across demographic groups, thus possibly leading to less stable/effective learning (added complexity to the loss might also be a negative that worsens with the number of groups).

8 Conclusion

We conduct the first investigation of how data augmentation via interpolation affects MC-based fairness on multiple minority groups of multiple sizes for binary tabular data classification. We find that while Fair Mixup is not so fair on multiple groups, regular mixup mitigates MC violations across many groups, both by itself and together with MC post-processing. Our investigation opens several avenues of future work, with our evaluation pipeline being easily extensible to data augmentation on probabilistic fairness in other modalities (e.g. vision, language) and ML problems (e.g. continuous/categorical labels).

Ethical Statement

Augmentation introduces synthetic data and alters demographic representation to present the illusion that certain groups are well-represented. We urge creators and users of augmented datasets to be transparent about augmentation methods used. We lead by example as we release our datasets with full methodological descriptions. Furthermore, we caution that our implemented augmentation methods can substantially alter outcomes in real-world decision-making settings, and examining multicalibration is meant to be a supplement to, not a replacement for, frameworks addressing binary, individual-level fairness.

Acknowledgements

We thank Professor Cynthia Dwork and Pranay Tankala for teaching the course that inspired this work and for mentoring us through the initial stages of the project. We also thank Professor Maarten Sap and Alfredo Gomez for helpful feedback during the paper writing and rebuttal process.

A Dataset Details

Table 4 summarizes the input features of the EMPLOYMENT dataset, and Table 5 summarizes the features for INCOME.

A.1 Hyperparameter Search

We explore a few choices of d , k , and λ for our MIXUP and Fair Mixup implementations. For Fair Mixup, we try each of the 24 combinations of $d \in \{10, 55, 100\}$, $k \in \{1, 3, 40, 100\}$, and $\lambda \in \{0.25, 0.5\}$ on the subsets of each prediction task from California from the year 2022 (while running these combinations on all 40 subsets would be ideal, this one search took us over a week and thus would be intractable to replicate). We determine the best (d, k, λ) triple for each (dataset, method) combination and use that triple on the other 39 (state, year) subsets for that dataset and method. We measure “best” via the highest average of (1) percent increase in balanced accuracy over BASE and (2) percent decrease in mean individual-group MC violation over BASE.

Feature	Code	Type	Scale
Detailed race code	RAC3P	Categorical	100
Relationship status	RELP (2018)	Categorical	18
Relationship status	RELSHIP (2019-2022)	Categorical	19
Mobility status	MIG	Categorical	4
Military service	MIL	Categorical	5
Ancestry	ANC	Categorical	4
Employment status of parents	ESP	Categorical	9
Citizenship status	CIT	Categorical	5
Marital status	MAR	Categorical	5
Cognitive difficulty	DREM	Categorical	3
Age	AGEP	Continuous	0-99
Educational attainment	SCHL	Continuous	0-24
Sex	SEX	Binary	-
Hearing difficulty	DEAR	Binary	-
Vision difficulty	DEYE	Binary	-
Born in US	NATIVITY	Binary	-
Disability	DIS	Binary	-

Table 4: Information about input features selected for EMPLOYMENT. Note on Scale column: for categorical variables, the Scale indicates the number of categories for the variable. For continuous variables, the Scale indicates the minimum and maximum of the range for the variable.

For MIXUP, we fix $d = 10$, state = CA, and year = 2022 and search over the 8 combinations of $k \in \{1, 3, 40, 100\}$ and $\lambda \in \{0.25, 0.5\}$, as $d = 55$ and $d = 100$ always dramatically worsened both efficiency and performance in our hyperparameter search for Fair Mixup. We measure and determine the best (k, λ) for each (dataset, method) tuple as we do in our Fair Mixup search, and we apply these hyperparameters across all states and years within each dataset and method.

One final hyperparameter is determining whether to create batches by sampling without replacement or by sampling the smaller group with replacement until it is equal to the specified batch size. We experiment with both options on California $\times$ 2022 $\times$ ALL for all FM methods and determine that the best performance and fairness results from the following choices: (1) if the group is smaller than 183617.4 (approximately 0.25% of the EMPLOYMENT dataset size, multiplied by the training split percentage) and the fairness metric is demographic parity or multiaccuracy, then we take a sample of size b , with replacement if b is bigger than the size of the group, without replacement otherwise; (2) otherwise, we take a sample of size $\min(\text{group size}, b)$, without replacement.

Our final hyperparameter selections are in Table 6.

Feature	Code	Type	Scale
Detailed race code	RAC3P	Categorical	100
Class of worker	COW	Categorical	10
Marital status	MAR	Categorical	5
Occupation	OCCP	Categorical	531
Relationship status	REL (2018)	Categorical	18
Relationship status (2019-2022)	RELSHIP	Categorical	19
Place of birth	POBP	Categorical	223
Age	AGEP	Continuous	0 - 99
Educational attainment	SCHL	Continuous	0 - 24
Usual hours worked per week in last 12 months	WKHP	Continuous	1 - 98
Sex	SEX	Binary	-
Disability	DIS	Binary	-

Table 5: Information about input features selected for our INCOME datasets from `folktables`. Note on Scale column: for categorical variables, the Scale indicates the number of categories for the variable. For continuous variables, the Scale indicates the minimum and maximum of the range for the variable.

Dataset	Method	d	k	λ
EMPLOYMENT	MIXUP	10	3	0.25
	MIXUP _{EO}	10	100	0.25
	MIXUP _{MA}	10	3	0.25
	MIXUP _{MC}	10	40	0.25
	FM _{DP}	10	100	0.5
	FM _{EO}	10	100	0.25
	FM _{MA}	10	100	0.25
	FM _{MC}	10	100	0.5
INCOME	MIXUP	10	40	0.25
	MIXUP _{EO}	10	40	0.5
	MIXUP _{MA}	10	40	0.25
	MIXUP _{MC}	10	40	0.5
	FM _{DP}	10	3	0.25
	FM _{EO}	10	3	0.5
	FM _{MA}	10	3	0.5
	FM _{MC}	10	3	0.25

Table 6: Hyperparameters d , k , and λ used for each (Fair) Mixup method and dataset.

B Additional Results

Table 7 gives the raw balanced accuracy percentages that contribute to the percent changes shown in Table 3, while Table 8 gives the raw worst-group MC α s that contribute to those same percent changes. Our full suite of summary statistics of results can be found at <http://tiny.cc/mfm-results>. Each number in the spreadsheet represents a mean over ten trials (random seeds 0 through 9).

C Implementation Details

C.1 Neural Network

The NN we used for our experiments was directly taken from Chuang and Mroueh (2021). It consists of 3 hidden layers of size 200 using ReLU activation after the first and second layers, and an output layer of size 1 with a sigmoid activation. We use the Adam optimizer with learning rate 0.001, as in Chuang and Mroueh (2021) as well. We use binary cross-entropy loss and train for 10 epochs with $n = 100$ iterations of mini-batch selection ($b = 500$) each. We use the PyTorch¹ library.

C.2 (Fair) Mixup

For every mini-batch in the Mixup and Fair Mixup variants, we generate a fresh $t \sim \text{Beta}(\epsilon, \epsilon)$, with $\epsilon = 1$, as in Chuang and Mroueh (2021). Also following Chuang and Mroueh (2021), even though the integral goes continuously from 0 to 1 in the mathematical specification of Fair Mixup, we only generate one interpolated point (t of the way from the minority group member to the majority group member of each pair) per pair due to practical time and compute power constraints.

C.3 Post-Processing Algorithms

To enforce MC, we use $d = 10$, and for both MA and MC enforcement, we use $\alpha = 0.01$ (strictly smaller than the smallest mean MC violation among the results of the post-processing-free methods). We require α -calibration to hold on the entirety of S rather than just a $(1 - \alpha)$ fraction of S , as enforcing the latter definition would naively require us to calculate the calibration of the predictor w.r.t. each of the $\binom{|S|}{(1-\alpha)|S|}$ subsets of S and terminate the algorithm if and only if the predictor was α -calibrated on the entirety of at least one of these sets, which would take an unreasonable amount of time.

We treat the predictor as an array-like object of probabilities $\in [0, 1]$ where the i th element of the list corresponds to f_i . While pseudocode for multicalibrating a predictor is provided in Hebert-Johnson et al. (2018), it does not account for practical implementation considerations, most importantly the additional output of a list of rules specifying what updates to apply to predictions on future unseen data. Thus, we provide code for the MC algorithm here and in our public repo (to be released upon publication). The inputs to the algorithm are p , the initial predictor’s outputs; $\mathcal{C}$, the set of subsets of the population that need to be multicalibrated; y , a list of true outcomes in $\{0, 1\}$; α , the violation parameter, and (for MC only) d , the prediction interval discretization parameter.

For prediction adjustments learned from the post-processing set to be useful, they must be applied to a baseline predictor at *test time*. To achieve this, we append the updates made to our post-processing sets to a `list` and return the list at the end of the algorithms. We then apply the updates in the list one at a time to the predictors at test time. This mimics the conceptualization of the MC algorithm as a circuit, as

¹<https://pytorch.org/>

Method	EMPLOYMENT					INCOME				
	ALL	BIG	SMALL	DIS	DLFR	ALL	BIG	SMALL	DIS	DLFR
BASE	82.37	82.37	82.37	82.37	82.37	79.82	79.82	79.82	79.82	79.82
FAIRBASE	81.96	82.05	82.01	82.33	82.19	78.94	78.99	79.04	78.93	79.08
MIXUP	81.64	81.64	81.64	81.64	81.64	79.67	79.67	79.67	79.67	79.67
MIXUP _{EO}	81.96	82.05	81.87	82.30	82.19	78.95	78.95	78.96	78.87	79.06
MIXUP _{MA}	81.95	82.04	81.93	82.27	82.07	78.90	78.90	78.93	78.91	78.93
MIXUP _{MC}	81.99	82.06	81.93	82.29	82.05	78.89	78.91	78.92	78.94	78.88
FM _{DP}	81.98	82.05	81.95	82.27	82.09	78.86	78.94	78.93	78.90	79.04
FM _{EO}	81.96	82.04	81.89	82.31	82.18	78.93	78.97	78.95	78.93	79.10
FM _{MA}	81.97	82.06	81.91	82.29	82.16	78.90	78.92	78.98	78.92	78.91
FM _{MC}	81.97	82.05	81.81	82.28	82.05	78.90	78.91	78.91	78.90	78.94
ENFORCE _{MA}	82.35	82.39	82.31	82.39	82.32	79.60	79.67	79.64	79.65	79.52
ENFORCE _{MC}	82.14	82.18	82.25	82.33	82.28	79.30	79.27	79.61	79.64	79.46
MIXUP _{ENFORCE_{MC}}	81.57	81.65	81.51	81.77	81.64	79.22	79.27	79.56	79.58	79.45

Table 7: Balanced accuracies (% , mean of 10 trials) of methods across 40 (state, year) pairs $\times$ EMPLOYMENT and INCOME.

Method	EMPLOYMENT					INCOME				
	ALL	BIG	SMALL	DIS	DLFR	ALL	BIG	SMALL	DIS	DLFR
BASE	0.674	0.639	0.660	0.121	0.609	0.668	0.666	0.263	0.109	0.576
FAIRBASE	0.705	0.628	0.654	0.122	0.602	0.656	0.650	0.307	0.133	0.640
MIXUP	0.628	0.638	0.661	0.121	0.610	0.668	0.665	0.246	0.099	0.555
MIXUP _{EO}	0.703	0.633	0.659	0.120	0.606	0.661	0.658	0.298	0.132	0.636
MIXUP _{MA}	0.713	0.608	0.635	0.126	0.587	0.639	0.630	0.303	0.131	0.649
MIXUP _{MC}	0.708	0.572	0.602	0.136	0.557	0.603	0.592	0.302	0.130	0.640
FM _{DP}	0.715	0.642	0.667	0.117	0.614	0.669	0.667	0.307	0.128	0.655
FM _{EO}	0.700	0.638	0.662	0.119	0.610	0.666	0.663	0.295	0.131	0.630
FM _{MA}	0.709	0.621	0.648	0.124	0.598	0.649	0.640	0.305	0.132	0.651
FM _{MC}	0.706	0.593	0.622	0.131	0.574	0.626	0.614	0.303	0.131	0.630
ENFORCE _{MA}	0.679	0.537	0.566	0.145	0.522	0.570	0.555	0.268	0.114	0.591
ENFORCE _{MC}	0.559	0.482	0.530	0.161	0.486	0.522	0.504	0.250	0.147	0.478
MIXUP _{ENFORCE_{MC}}	0.526	0.474	0.519	0.176	0.484	0.509	0.475	0.233	0.132	0.422

Table 8: MC α of least calibrated group for methods across 40 (state, year) pairs $\times$ EMPLOYMENT and INCOME.

suggested in Hebert-Johnson et al. (2018), where a predictor achieves MC on more and more sets as it passes through increasingly large computational gates until it is multicalibrated with respect to the entire superset.

D Full Fair Mixup Specification

In all formulae below, $S' = \neg S$.

D.1 FM_{DP}

We begin with the metric specification for demographic parity:

$$\mathcal{M}_S = \Delta DP_S(f) = |\mathbb{E}_{x_i \sim S}[f_i] - \mathbb{E}_{x_i \sim S'}[f_i]|. \quad (9)$$

For a continuously differentiable function $T : \mathcal{X}^2 \times [0, 1] \rightarrow \mathcal{X}$ such that $T(x_1, x'_1, 0) = x_1$ and $T(x_1, x'_1, 1) = x'_1$, this metric applied to the interpolated synthetic points would be

$$\Delta DP_S(f) = \left| \int_0^1 \frac{d}{dt} \int f(T(x_1, x'_1, t)) dS(x_1) dS'(x'_1) dt \right|. \quad (10)$$

We refer the reader to Chuang and Mroueh (2021) for a full proof. The gradient of this metric would be

$$\mathcal{R}_{\text{mixup}}^{\text{DP}_S} = \int_0^1 \left| \int \mathcal{I} dS(x_1) dS'(x'_1) dt \right|, \quad (11)$$

where

$$\mathcal{I} = \langle \nabla_x f(T(x_1, x'_1, t)), x_1 - x'_1 \rangle. \quad (12)$$

D.2 FM_{EO}

Again, we begin with the metric definition:

$$\mathcal{M}_S = \Delta EO_S(f) = \sum_{y \in \{0,1\}} |\mathbb{E}_{x_i \sim S_y}[f_i] - \mathbb{E}_{x_i \sim S'_y}[f_i]|, \quad (13)$$

where $S_y = \{x_i \in S : y_i = y\}$, and $S' = \neg S$. Then, we transform it into an integral to be able to apply it to interpolated points:

$$\Delta EO_S(f) = \sum_{y \in \{0,1\}} \left| \int_0^1 \frac{d}{dt} \int f_T dS_y(x_1) dS'_y(x'_1) dt \right|, \quad (14)$$

where

$$f_T = f(T(x_1, x'_1, t)). \quad (15)$$

Next, we take the Jacobian to get our per-group penalty:

$$\mathcal{R}_{\text{mixup}}^{\text{EO}_S} = \sum_{y \in \{0,1\}} \int_0^1 \left| \int \mathcal{I} dS_y(x_1) dS'_y(x'_1) dt \right|, \quad (16)$$

with $\mathcal{I}$ defined as in Equation 12.

D.3 FM_{MA}

We begin with our metric specification for MA:

$$\mathcal{M}_S = \Delta MA_S(f) = |\mathbb{E}_{x_i \sim S}[e_i] - \mathbb{E}_{x_i \sim S'}[e_i]|, \quad (17)$$

Listing 1: Implementation of MC post-processing.

```
1  # Function to calculate MC violations
2  # Inputs:
3      # p: list of predicted probabilities
4      # S: list of indices specifying group to consider
5      # y: list of true outcomes
6      # d: group discretization parameter
7  # Outputs:
8      # v: index of prediction interval with highest violation
9      # highest: value of highest violation at interval v
10     # S_alphas: list of violations for all prediction intervals
11     # S_vs: list where vth element is the subset of S with predictions in interval v
12 def calculate_calibration(p, S, y, d):
13     # split S into intervals
14     S_vs = [[] for _ in range(d + 1)]
15     for i in range(d + 1):
16         for e in S:
17             if e is not None and i / d <= p[e] < (i + 1) / d:
18                 S_vs[i].append(e)
19     # compute violations
20     S_alphas = [0 for _ in range(d + 1)]
21     for interval in range(len(S_vs)):
22         n = len(S_vs[interval])
23         if n != 0:
24             predictor_sum = 0.
25             y_sum = 0.
26             for e in S_vs[interval]:
27                 if e is not None:
28                     predictor_sum += p[e]; data_sum += y[e]
29             S_alphas[interval] = (y_sum - predictor_sum) / n
30     # return the violation posed by the interval with the maximum violation
31     v_max, v_min = S_alphas.index(max(S_alphas)), S_alphas.index(min(S_alphas))
32     v = v_max if abs(S_alphas[v_max]) > abs(S_alphas[v_min]) else v_min
33     highest = abs(S_alphas[v])
34     return v, highest, S_alphas, S_vs
35
36 # Function to enforce max MC violation of alpha w.r.t. C on post-processing set X with
37 # predictions p (discretized by parameter d) and true outcomes y
38 # Outputs:
39     # p_new: updated predictions on post-processing set
40     # updates: list of rules mapping (group, interval) pairs to future prediction
41     # adjustments
42 def multicalibrate(X, p, C, y, alpha, d):
43     p_new, done, seen, updates = p.copy(), False, [], []
44     while not done:
45         # randomly select 1 group at a time to calibrate
46         ind = np.random.choice(len(C), size = 1, replace = False)[0]
47         A = np.ones(X.shape[0])
48         for _, criterion, value in C[ind]:
49             A *= X[:, criterion] == value
50             S = np.where(A == 1)[0]; seen.append(ind)
51             # if X has members of S, calculate and adjust prediction errors
52             if len(S) > 0:
53                 v, highest, S_alphas, S_vs = calculate_calibration(p_new, S, y, d)
54                 # if violation too high, update p by nudging
55                 if (highest > alpha):
56                     seen = []
57                     updates.append((C[ind], v, S_alphas[v]))
58                     for el in S_deciles[j]:
59                         if e is not None:
60                             p_new[e] += S_alphas[v]
61                             p_new[e] = min(p_new[e], 1); p_new[e] = max(p_new[e], 0)
62             done = len(seen) == len(C)
63     return p_new, updates
```

Listing 2: Implementation of MA post-processing.

```

1  # Function to calculate MA violations
2  # Inputs:
3      # p: list of predicted probabilities
4      # S: list of indices specifying group to consider
5      # y: list of true outcomes
6  # Output: mean prediction error across elements of S
7  def calculate_accuracy(p, S, y):
8      predictor_sum = 0
9      y_sum = 0
10     # compute violations
11     for e in S:
12         if e is not None:
13             predictor_sum += p[e]; y_sum += y[e]
14     return (y_sum - predictor_sum) / len(S)
15
16 # Function to enforce max MA violation of alpha w.r.t. C on post-processing set X with
    # predictions p and true outcomes y
17 # Outputs:
18     # p_new: updated predictions on post-processing set
19     # updates: list of rules mapping groups to future prediction adjustments
20 def enforce_multiaccuracy(X, p, C, y, alpha):
21     p_new, done, seen, updates = p.copy(), False, [], []
22     while not done:
23         # randomly choose 1 set to start with
24         ind = np.random.choice(len(C), size = 1, replace = False)[0]
25         A = np.ones(X.shape[0])
26         for _, criterion, value in C[ind]:
27             A *= X[:, criterion] == value
28         S = np.where(A == 1)[0]
29         seen.append(ind)
30         # if X has members of S, calculate and adjust errors
31         if len(S) > 0:
32             violation = calculate_accuracy(p_new, S, y)
33             # if violation too high, update p by nudging
34             if (abs(violation) > alpha):
35                 seen = []
36                 updates.append((C[ind], violation))
37                 for e in S:
38                     if e is not None:
39                         p_new[e] += violation
40                         p_new[e] = min(p_new[e], 1); p_new[e] = max(p_new[e], 0)
41         done = len(seen) == len(C):
42     return p_new, updates

```

Transforming this into an integral, we obtain:

$$\Delta \text{MA}_S(f) = \left| \int_0^1 \frac{d}{dt} (\mu_f(t) - \mathbb{E}_{(x_1, x'_1) \sim (S, S')} [y(t)]) dt \right|, \quad (18)$$

where

$$\mu_f(t) = \mathbb{E}_{(x_1, x'_1) \sim (S, S')} [f(tx_1 + (1-t)x'_1)], \quad (19)$$

and

$$y(t) = ty_1 + (1-t)y'_1. \quad (20)$$

Taking the Jacobian, we have

$$R_{\text{mixup}}^{\text{MA}_S} = \int_0^1 \left| \int \langle \nabla_x \mathcal{T}, x_1 - x'_1 \rangle dS(x_1) dS'(x'_1) \right| dt, \quad (21)$$

where

$$\mathcal{T} = f(tx_1 + (1-t)x'_1) - (ty_1 + (1-t)y'_1). \quad (22)$$

D.4 FM_{MC}

Beginning with our metric definition:

$$\mathcal{M}_S = \Delta \text{MC}_S(f) = \sum_{v=0}^d |\mathbb{E}_{x_i \sim S_v^d} [e_i] - \mathbb{E}_{x_i \sim S_v'^d} [e_i]|, \quad (23)$$

where $S_v^d = \{x_i \in S : f_i \in [\frac{v}{d}, \frac{v+1}{d}]\}$, we create our integral as follows:

$$\Delta \text{MC}_{S_v^d} = \left| \int_0^1 \frac{d}{dt} (\mu_f(t) - \mathbb{E}_{(x_1, x'_1) \sim (S_v^d, S_v'^d)} [y(t)]) dt \right|, \quad (24)$$

where

$$\mu_f(t) = \mathbb{E}_{x_1, x'_1 \sim S_v^d, S_v'^d} [f(tx_1 + (1-t)x'_1)], \quad (25)$$

with $y(t)$ as in Eq. 20. Then,

$$\mathcal{R}_{\text{mixup}}^{\text{MC}_{S_v^d}} = \int_0^1 \left| \int \langle \nabla_x \mathcal{T}, x_1 - x'_1 \rangle dS_v^d(x_1) dS_v^{td}(x'_1) \right| dt, \quad (26)$$

with $\mathcal{T}$ as in Eq. 22. Finally, we sum over the prediction intervals to get

$$\mathcal{R}_{\text{mixup}}^{\text{MC}_S} = \sum_{v=0}^d \mathcal{R}_{\text{mixup}}^{\text{MC}_{S_v^d}}. \quad (27)$$

References

- Allison, K.; Patel, D.; and Kaur, R. 2022. Assessing multiple factors affecting minority participation in clinical trials: Development of the clinical trials participation barriers survey. *Cureus*, 14(4): e24424.
- Barda, N.; Riesel, D.; Akriv, A.; Levy, J.; Finkel, U.; Yona, G.; Greenfeld, D.; Sheiba, S.; Somer, J.; Bachmat, E.; Rothblum, G. N.; Shalit, U.; Netzer, D.; Balicer, R.; and Dagan, N. 2020. Developing a COVID-19 mortality risk prediction model when individual-level data are not available. *Nature Communications*, 11(1): 4439.
- Becker, B.; and Kohavi, R. 1996. Census Income Data Set.
- Błasiok, J.; Gopalan, P.; Hu, L.; Kalai, A. T.; and Nakkiran, P. 2024. Loss Minimization Yields Multicalibration for Large Neural Networks. In Guruswami, V., ed., *15th Innovations in Theoretical Computer Science Conference, ITCS 2024, January 30 to February 2, 2024, Berkeley, CA, USA*, volume 287 of *LIPIcs*, 17:1–17:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.
- Błasiok, J.; Gopalan, P.; Hu, L.; and Nakkiran, P. 2023. When Does Optimizing a Proper Loss Yield Calibration? In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Chawla, N. V.; Bowyer, K. W.; Hall, L. O.; and Kegelmeyer, W. P. 2002. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16: 321–357.
- Chouldechova, A. 2017. Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments. *Big Data*, 5(2): 153–163. PMID: 28632438.
- Chuang, C.-Y.; and Mroueh, Y. 2021. Fair Mixup: Fairness via Interpolation. In *International Conference on Learning Representations*.
- Ding, F.; Hardt, M.; Miller, J.; and Schmidt, L. 2021. Retiring Adult: New Datasets for Fair Machine Learning. In Beygelzimer, A.; Dauphin, Y.; Liang, P.; and Vaughan, J. W., eds., *Advances in Neural Information Processing Systems*.
- Foygel Barber, R.; Candès, E. J.; Ramdas, A.; and Tibshirani, R. J. 2020. The limits of distribution-free conditional predictive inference. *Information and Inference: A Journal of the IMA*, 10(2): 455–482.
- Google. 2024. Data Commons.
- Gopalan, P.; Okoroafor, P.; Raghavendra, P.; Shetty, A.; and Singhal, M. A. 2024. Omnipredictors for Regression and the Approximate Rank of Convex Functions. In *COLT*.
- Guo, L. N.; Lee, M. S.; Kassamali, B.; Mita, C.; and Nambudiri, V. E. 2022. Bias in, bias out: Underreporting and underrepresentation of diverse skin types in machine learning research for skin cancer detection-A scoping review. *J. Am. Acad. Dermatol.*, 87(1): 157–159.
- Hansen, D.; Devic, S.; Nakkiran, P.; and Sharan, V. 2024. When is Multicalibration Post-Processing Necessary? arXiv:2406.06487.
- Hardt, M.; Price, E.; and Srebro, N. 2016. Equality of opportunity in supervised learning. In *Proceedings of the 30th International Conference on Neural Information Processing Systems, NIPS’16*, 3323–3331. Red Hook, NY, USA: Curran Associates Inc. ISBN 9781510838819.
- Hebert-Johnson, U.; Kim, M.; Reingold, O.; and Rothblum, G. 2018. Multicalibration: Calibration for the (Computationally-Identifiable) Masses. In Dy, J.; and Krause, A., eds., *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, 1939–1948. PMLR.
- Huang, J.; Galal, G.; Etemadi, M.; and Vaidyanathan, M. 2022. Evaluation and mitigation of racial bias in clinical machine learning models: Scoping review. *JMIR Med. Inform.*, 10(5): e36388.
- Iosifidis, V.; and Ntoutsi, E. 2018. Dealing with Bias via Data Augmentation in Supervised Learning Scenarios. *Jo Bates Paul D. Clough Robert Jäschke*, 24.
- Jung, C.; Lee, C.; Pai, M.; Roth, A.; and Vohra, R. 2021. Moment Multicalibration for Uncertainty Estimation. In Belkin, M.; and Kpotufe, S., eds., *Proceedings of Thirty Fourth Conference on Learning Theory*, volume 134 of *Proceedings of Machine Learning Research*, 2634–2678. PMLR.
- Jung, C.; Noarov, G.; Ramalingam, R.; and Roth, A. 2023. Batch Multivalid Conformal Prediction. In *International Conference on Learning Representations*.
- Kim, M. P.; Ghorbani, A.; and Zou, J. 2019. Multiaccuracy: Black-Box Post-Processing for Fairness in Classification. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society, AIES ’19*, 247–254. New York, NY, USA: Association for Computing Machinery. ISBN 9781450363242.
- Kumar, A.; Sarawagi, S.; and Jain, U. 2018. Trainable Calibration Measures for Neural Networks from Kernel Mean Embeddings. In Dy, J.; and Krause, A., eds., *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, 2805–2814. PMLR.
- Sharma, S.; Zhang, Y.; Ríos Aliaga, J. M.; Bouneffouf, D.; Muthusamy, V.; and Varshney, K. R. 2020. *Data Augmentation for Discrimination Prevention and Bias Disambiguation*, 358–364. New York, NY, USA: Association for Computing Machinery. ISBN 9781450371100.
- Wadhwa, M.; Bhambhani, M.; Jindal, A.; Sawant, U.; and Madhavan, R. 2022. Fairness for Text Classification Tasks with Identity Information Data Augmentation Methods.
- Wald, Y.; Feder, A.; Greenfeld, D.; and Shalit, U. 2021. On Calibration and Out-of-Domain Generalization. In Beygelz-

Immer, A.; Dauphin, Y.; Liang, P.; and Vaughan, J. W., eds., *Advances in Neural Information Processing Systems*.

Yucer, S.; Akcay, S.; Moubayed, N. A.; and Breckon, T. 2020. Exploring racial bias within face recognition via per-subject adversarially-enabled data augmentation. In *Computer Vision and Pattern Recognition Workshops*. IEEE. To be presented at the Workshop on Fair, Data Efficient and Trusted Computer Vision.

Zhang, H.; Cissé, M.; Dauphin, Y. N.; and Lopez-Paz, D. 2017. mixup: Beyond Empirical Risk Minimization. *CoRR*, abs/1710.09412.